

Divide y vencerás

Análisis y rendimiento de algoritmos

**Iván Mansilla, Diego Peralta,
Catalina Viñas**

Diseño de Algoritmos

Universidad de Magallanes



Docente: MsC. Jacqueline
Aldridge

Marco teórico

1. División

Descomponer el problema central en subproblemas independientes más pequeños

2. Conquista

Resolver cada subproblema de forma recursiva hasta alcanzar el caso base

3. Combinación

Fusionar las soluciones de los subproblemas para construir respuesta final

Merge Sort

Divide recursivamente el arreglo en dos mitades y luego las fusiona usando arreglos auxiliares.

Optimización: Incorpora un umbral threshold que cambia a Insertion Sort para subarreglos pequeños

Quick Sort

Divide el arreglo con partición de Lomuto usando un pivote y ordena recursivamente ambos lados.

Variantes de pivote: Primero, ultimo, aleatorio y mediana.

Búsqueda binaria recursiva

Acorta rango de búsqueda a la mitad

Búsqueda por interpolación

Utiliza la formula $pos = low + \frac{(high - low)}{(arr[high] - arr[low])} * (x - arr[low])$ para estimar la posición del elemento buscado en lugar de dividir el arreglo en mitades.

Búsqueda exponencial

Duplica el límite bound $\ast= 2$ hasta hallar el rango, luego aplica búsqueda binaria.

Búsqueda por rango

Usa búsqueda binaria recursiva para encontrar el primer y último índice con el valor objetivo.

Quick Select

Encuentra el k -ésimo elemento sin ordenar completamente usando partición recursiva. Selecciona pivote mediante mediana de 3 y descarta mitades innecesarias.

Complejidades teóricas

Marco teórico

Implementación

Resultados

Conclusiones

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Merge Sort Optimizado	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort (último elemento)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Quick Sort (primer elemento)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Quick Sort (elemento aleatorio)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Quick Sort (mediana de tres)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Búsqueda Binaria Recursiva	$O(\log n)$	$O(\log n)$	$O(\log n)$
Búsqueda por Interpolación	$O(1)$	$O(\log \log n)$	$O(n)$
Búsqueda Exponencial	$O(1)$	$O(\log i)$	$O(\log n)$
Búsqueda por Rango	$O(\log n)$	$O(\log n + k)$	$O(n)$
Quick Select	$O(n)$	$O(n)$	$O(n^2)$

Implementación

Benchmarks

Marco teórico
Implementación
Resultados
Conclusiones

N deportistas creciente con 50 pasos, 5 repeticiones cada caso (mejor, promedio y peor). Se guardan los promedios en archivos CSV.

Search Benchmark

Compara 6 algoritmos de búsqueda en mejor, promedio y peor caso con tamaños crecientes.

Selection Benchmark

Mide Quick Select en mejor y peor caso para encontrar el k -ésimo elemento.

Sort Benchmark

Evalúa 10 variantes de ordenamiento con intervalos para identificar rendimiento relativo.

Threshold Benchmark

Prueba valores threshold de Merge Sort (2, 4, 8, ..., 128) para optimizar cambio a Insertion Sort.

Otras funcionalidades

Marco teórico

Implementación

Resultados

Conclusiones

Flujo interactivo

Permite elegir algoritmos de ordenamiento y visualizar resultados en tiempo real.

Búsqueda por ID

Localiza deportista mediante ID usando diferentes algoritmos de búsqueda.

Ranking Top N

Utiliza **Quick Select** para encontrar los N mejores deportistas por puntaje.

Rango de puntajes

Ordena con **Quick Sort (mediana)** y aplica **búsqueda por rango**.

k-ésimo deportista

Quick Select para encontrar el k-ésimo deportista sin ordenar.

Resultados

Sorting secuencial vs divide y vencerás

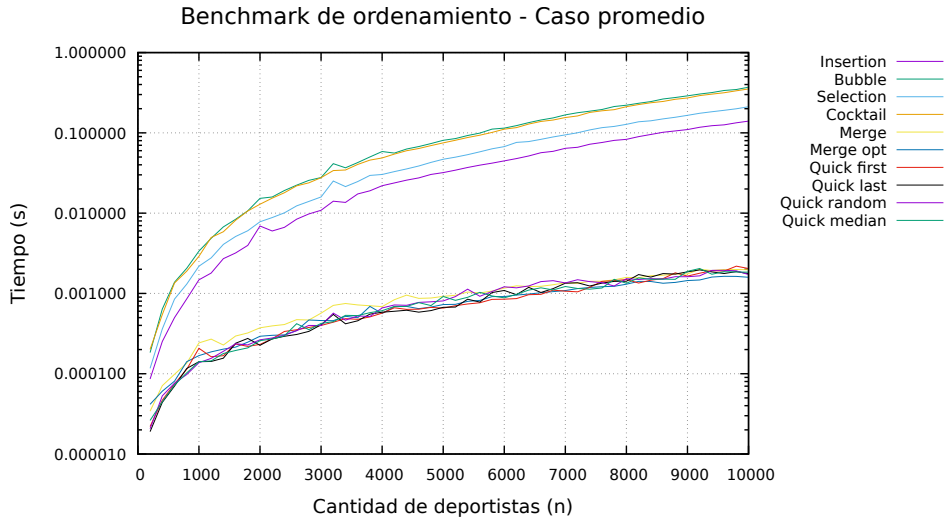


Marco teórico

Implementación

Resultados

Conclusiones



Sorting

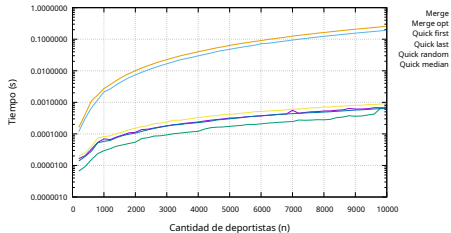
Marco teórico

Implementación

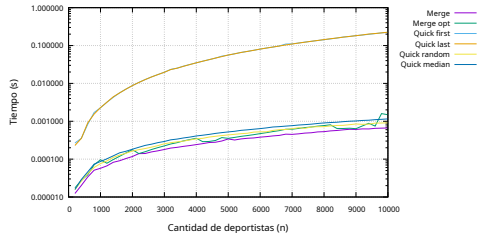
Resultados

Conclusiones

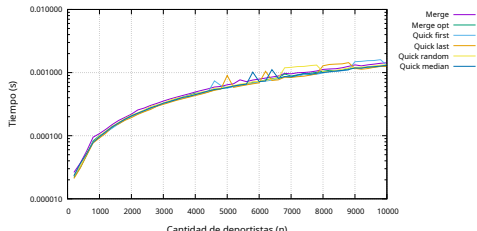
Benchmark de ordenamiento - Mejor caso



Benchmark de ordenamiento - Peor caso



Benchmark de ordenamiento - Caso promedio

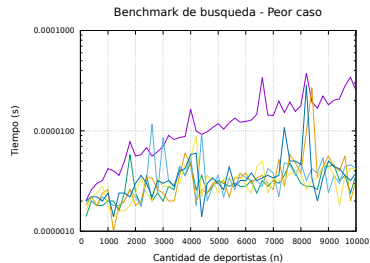
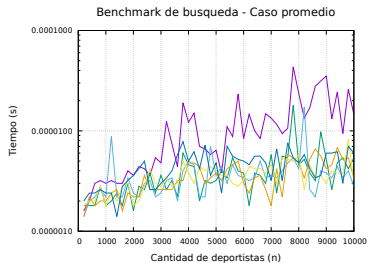


Marco teórico

Implementación

Resultados

Conclusiones



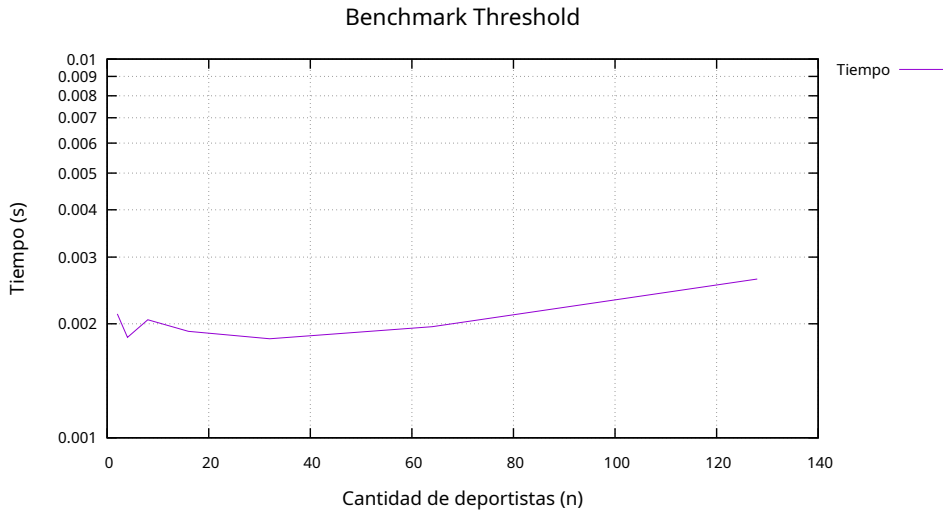
Threshold de Merge Sort

Marco teórico

Implementación

Resultados

Conclusiones



Conclusiones

Conclusiones

Marco teórico
Implementación
Resultados
Conclusiones

- ◇ Merge sort es el algoritmo más eficiente y previsible
- ◇ En quick sort la elección del pivote es crucial para el rendimiento
- ◇ La recursión no siempre es la mejor opción para arreglos pequeños como se ve en el benchmark de threshold